

Stat 437 Project 1

John Salmon (011745357)

General rule and information

You must show your work in order to get points. Please prepare your report according to the rubrics on projects that are given in the syllabus. In particular, please note that you need to submit codes that would have been used for your data analysis. Your report can be in .doc, .docx, .html or .pdf format.

The project will assess your skills in K-means clustering, Hierarchical clustering, Nearest-neighbor classifier, and discriminant analysis for classification, for which visualization techniques you have learnt will be used to illustrate your findings.

Data set and its description

Please download the data set “TCGA-PANCAN-HiSeq-801x20531.tar.gz” from the website <https://archive.ics.uci.edu/ml/machine-learning-databases/00401/>. A brief description of the data set is given at <https://archive.ics.uci.edu/ml/datasets/gene+expression+cancer+RNA-Seq>.

You need to decompress the data file since it is a .tar.gz file. Once uncompressed, the data files are “labels.csv” that contains the cancer type for each sample, and “data.csv” that contains the “gene expression profile” (i.e., expression measurements of a set of genes) for each sample. Here each sample is for a subject and is stored in a row of “data.csv”. In fact, the data set contains the gene expression profiles for 801 subjects, each with a cancer type, where each gene expression profile contains the gene expressions for the same set of 20531 genes. The cancer types are: “BRCA”, “KIRC”, “COAD”, “LUAD” and “PRAD”. In both files “labels.csv” and “data.csv”, each row name records which sample a label or observation is for.

Task A. Clustering

For this task, you need to apply k-means and hierarchical clustering to cluster observations into their associated cancer types, and report your findings scientifically and professionally. Your laptop may not have sufficient computational power to implement k-means and hierarchical clustering on the whole data set, and genes whose expressions are zero for most of the subjects may not be so informative of a cancer type.

Please use `set.seed(123)` for random sampling via the command `sample`, random initialization of `kmeans`, implementing the gap statistic, and any other process where artificial randomization is needed.

(Task A1) Complete the following data processing steps:

- Filter out genes (from “data.csv”) whose expressions are zero for at least 300 subjects, and save the filtered data as R object “gexp2”.
- Use the command `sample` to randomly select 1000 genes and their expressions from “gexp2”, and save the resulting data as R object “gexp3”.
- Use the command `sample` to randomly select 30 samples and their labels from the file “labels.csv”, and save them as R object “labels1”. For these samples, select the corresponding samples from “gexp3” and save them as R object “gexpProj1”.
- Use the command `scale` to standard the gene expressions for each gene in “gexpProj1”, so that they have sample standard deviation 1. Save the standardized data as R object “stdgexpProj1”.

```
set.seed(123)
# filter genes w/ expression zero for at least 300 subjects
# (aka columns with more than 300 rows containing 0)
num_zeros = colSums(gene_data == 0)
gexp2 = gene_data[, names(num_zeros[num_zeros < 300])]

# sample 1000 genes and their expressions
random_columns = sample(colnames(gexp2), 1000) #don't have to worry about sample column since
gexp3 = gexp2 %>%
  select(all_of(random_columns))

# Sample 30 samples and their labels from the file
# 'labels.csv'
sample_indices = sample(1:dim(gexp3)[1], size = 30, replace = FALSE)
gexpProj1 = gexp3[sample_indices, ]
gexpProj1 <- gexpProj1[order(rownames(gexpProj1)), ]

labels1 = gene_labels %>%
  filter(rownames(gene_labels) %in% rownames(gexpProj1))
labels1 = labels1 %>%
  mutate_at("Class", as.factor)

# scale the gene expressions
stdgexpProj1 = scale(gexpProj1)
```

(Task A2)

(Part 1 of Task A2) Randomly pick 50 genes and their expressions from “stdgexpProj1”, and do the following to these expressions: apply the “gap statistic” to estimate the number of clusters, apply K-means clustering with the estimated number of clusters given by the gap statistic, visualize the classification results using techniques given by “LectureNotes3_notes.pdf.pdf”, and provide a summary on classification errors. You may use the command `table` and “labels1” to obtain classification errors. Note that the cluster numbering given by `kmeans` will usually be coded as follows:

```
# Class label
# PRAD 5
# LUAD 4
# BRCA 1
# KIRC 3
# COAD 2
```

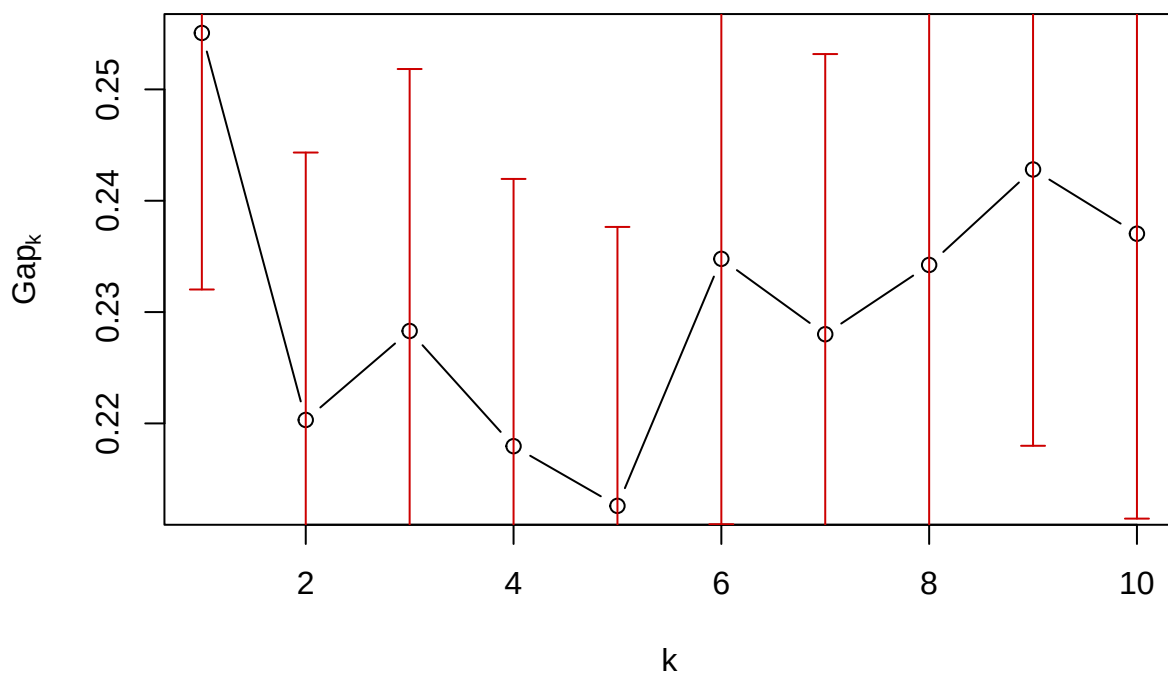
When you apply `clusGap`, please use arguments `K.max=10`, `B=200`, `iter.max=100`, and when you use `kmeans`, please use arguments `iter.max = 100`, `nstart=25`, `algorithm = c("Hartigan-Wong")`.

```
library(cluster)
library(ggplot2)
set.seed(123)
fiftygenes = sample(colnames(stdgexpProj1), 50)
fiftygeneset = stdgexpProj1[, fiftygenes]

gap = clusGap(fiftygeneset, FUNcluster = kmeans, K.max = 10,
              B = 200, iter.max = 100)

optimal_k = maxSE(gap$Tab[, "gap"], gap$Tab[, "SE.sim"], method = "Tibs2001SEmax")
plot(gap)
```

**clusGap(x = fiftygeneset, FUNcluster = kmeans, K.max = 10,
B = 200, iter.max = 100)**



```
cat("Clusgap Computed Optimal K value:", optimal_k, "\n")
```

```
## Clusgap Computed Optimal K value: 1
```

```
k_means = kmeans(fiftygeneset, optimal_k, iter.max = 100, nstart = 25,  
  algorithm = c("Hartigan-Wong"))
```

```
vis = data.frame(fiftygeneset)  
vis$Type = labels1 %>%  
  filter(rownames(labels1) %in% rownames(fiftygeneset)) %>%  
  pull(Class)  
vis$cluster = factor(k_means$cluster)
```

```
classification_table = table(vis$Type, vis$cluster)  
print(classification_table)
```

```
##  
##          1  
## BRCA 17  
## COAD  2  
## KIRC  4  
## LUAD  2  
## PRAD  5
```

```
correct_classifications = sum(diag(classification_table))  
total_classifications = sum(classification_table)  
classification_errors = total_classifications - correct_classifications  
error_rate = classification_errors/total_classifications
```

```
cat("Number of classification errors:", classification_errors,  
  "\n")
```

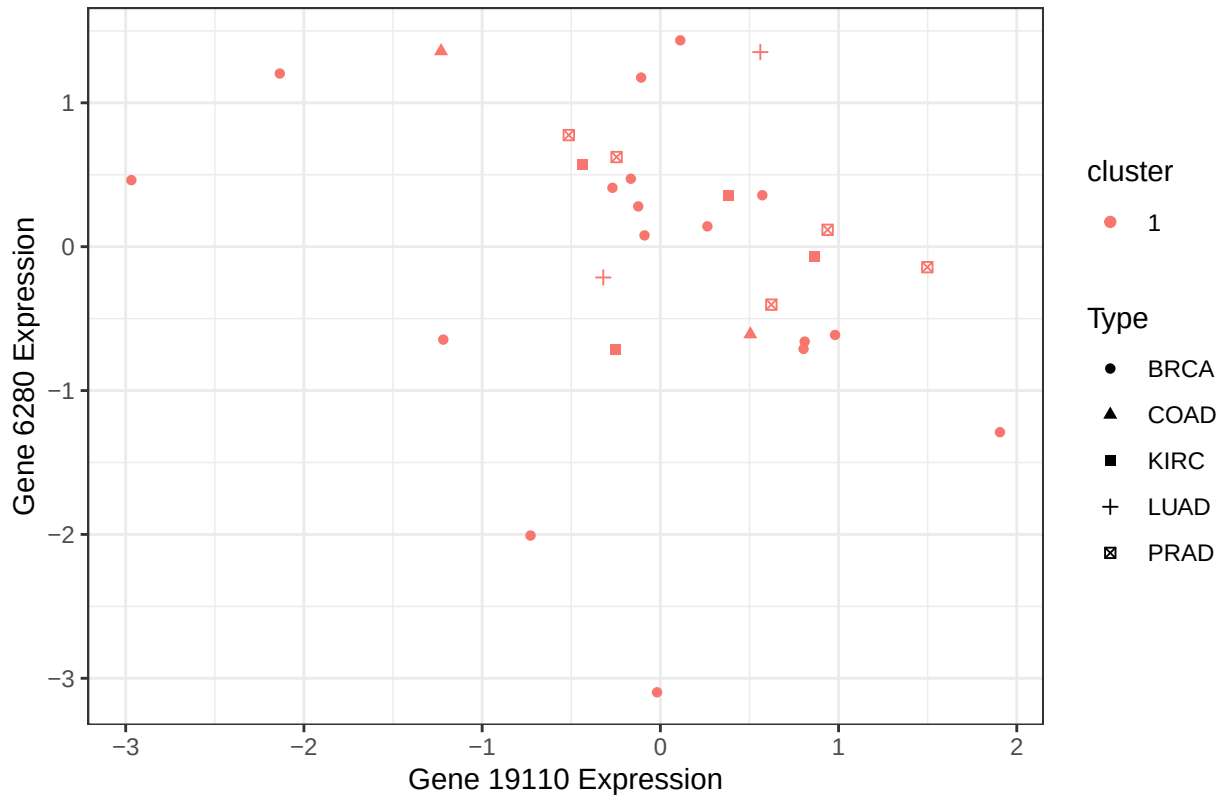
```
## Number of classification errors: 13
```

```
cat("Accuracy:", 1 - error_rate, "\n")
```

```
## Accuracy: 0.5666667
```

```
ggplot(vis, aes(x = gene_19110, y = gene_6280)) + xlab("Gene 19110 Expression") +  
  ylab("Gene 6280 Expression") + theme_bw() + geom_point(aes(shape = Type,  
    color = cluster)) + ggtitle("Clustering results using clusGap to estimate cluster number")
```

Clustering results using clusGap to estimate cluster number



The classification results from using `clusGap` to estimate an optimal number of clusters is not very good. `ClusGap` estimated that there should be only 1 cluster based on the genes from the 30 subjects used for calculation. Since we know what the true number of cluster labels should be, this is obviously incorrect since there are 5 different labels. The `k` means clustering made 14 out of 30 errors while classifying and had an accuracy of 0.56% which is fairly decent. However given the context of the way this classifier is assigning observations to a single label, this accuracy will vary wildly depending on the distribution of breast cancer occurrences in the data set it is given. This is because it is assigning every set of 50 gene expressions to the same cancer type: BRCA.

(**Part 2 of Task A2**) Upon implementing `kmeans` with k as the number of clusters, we will obtain the “total within-cluster sum of squares” $W(k)$ from the output `tot.withinss` of `kmeans`. If we try a sequence of $k = 1, 2, 3, \dots, 10$, then we get $W(k)$ for each k between 1 and 10. Let us look at the difference $\Delta_k = W(k) - W(k+1)$ for k ranging from 1 to 9. The K^* for which

$$\{\Delta_k : k < K^*\} \gg \{\Delta_k : k \geq K^*\}$$

is an estimate of the true number K of clusters in the data, where $\gg$ means “much larger”. Apply this method to obtain an estimate of K for the data you created in **Part 1**, and provide a plot of $W(k)$ against k for each k between 1 and 10. Compare this estimate with the estimate obtained in **Part 1** given by the gap statistic, comment on the accuracy of the two estimates, and explain why they are different.

```
set.seed(123)
n = 10
W = numeric(n)
```

```

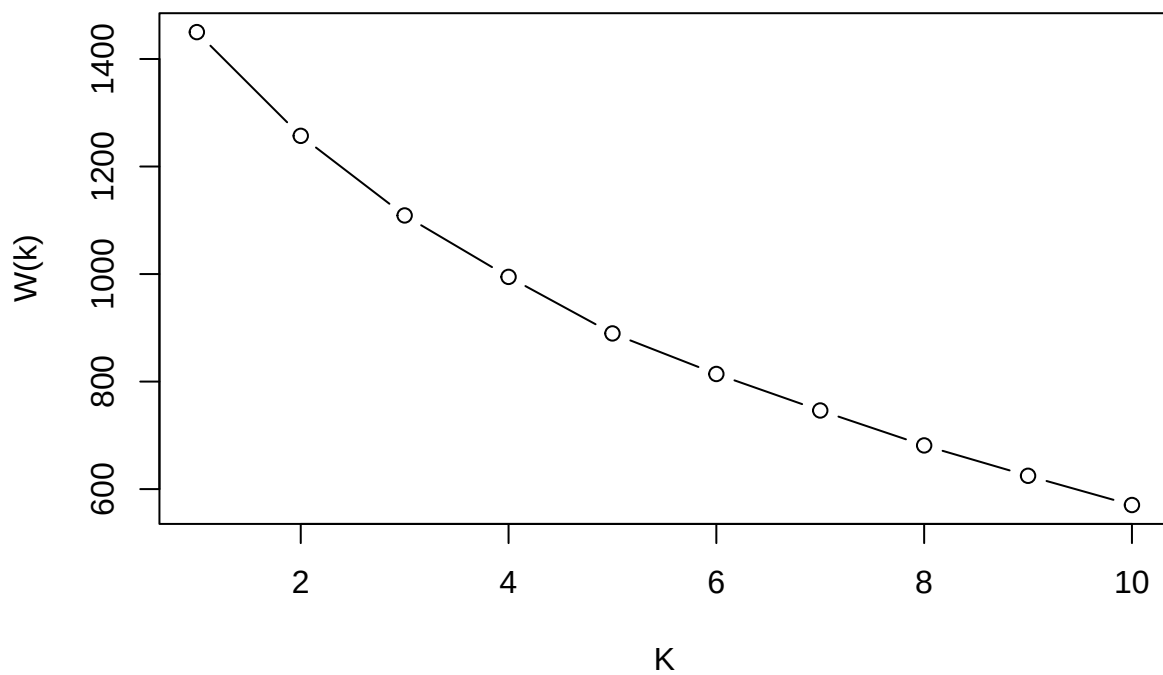
for (k in 1:n) {
  km = kmeans(fiftygeneset, k, iter.max = 100, nstart = 25,
    algorithm = c("Hartigan-Wong"))
  W[k] = km$tot.withinss
}

delta = numeric(n - 1)
for (i in 1:n - 1) delta[i] = W[i] - W[i + 1]

K_star = which.max(delta) + 1

plot(1:n, W, type = "b", xlab = "K", ylab = "W(k)")

```



```

cat("K* =", K_star, "\n")

```

```

## K* = 2

```

```

cat("ClusGap =", optimal_k, "\n")

```

```

## ClusGap = 1

```

```
k_means_star = kmeans(fiftygeneset, K_star, iter.max = 100, nstart = 25,
  algorithm = c("Hartigan-Wong"))
vis$cluster = factor(k_means_star$cluster) #overwrite previous cluster labels with new labels
classification_table = table(vis$Type, vis$cluster)
print(classification_table)
```

```
##
##      1  2
## BRCA  4 13
## COAD  1  1
## KIRC  2  2
## LUAD  2  0
## PRAD  0  5
```

```
correct_classifications = sum(diag(classification_table))
total_classifications = sum(classification_table)
classification_errors = total_classifications - correct_classifications
error_rate = classification_errors/total_classifications

cat("Number of classification errors:", classification_errors,
  "\n")
```

```
## Number of classification errors: 25
```

```
cat("Accuracy:", (1 - error_rate), "\n")
```

```
## Accuracy: 0.1666667
```

Both estimates are different from each other, the ClusGap estimate suggests that 1 cluster should be used, while the K-star method predicted 2 clusters. Examining the classification results after using k_means clustering with the K^* method's estimated number of clusters it is clear that while this estimate (2 labels) is closer to the truth (5 cancer types) the overall accuracy of the model has plummeted. With only 16% of the observations being classified correctly this clustering performance is not satisfactory.

(Part 3 of of Task A2) Randomly pick 250 genes and their expressions from “stdgexpProj1”, and for these expressions, do the analysis in **Part 1** and **Part 2**. Report your findings, compare your findings with those from **Part 1** and **Part 2**; if there are differences between these findings, explain why. Regard using more genes as using more features, does using more features necessarily give more accurate clustering or classification results?

```
set.seed(123)
twofiftygenes = sample(colnames(stdgexpProj1), 250)
twofiftygeneset = stdgexpProj1[, twofiftygenes]
```

Despite increasing the number of genes, there is not really a difference between the two clustering problems. Using `clusGap` to estimate a good k value to use still tells us to use the value 1, while using the k^* method tells us to use a value of 2. Since we know the true clustering labels, we know that these are not the correct values of k as there are 5 levels in `labels1`. In this application, providing the clustering algorithms with more features did not improve the quality or accuracy of our results.

(Task A3) Randomly pick 250 genes and their expressions from “`stdgexpProj1`”, and for these expressions, do the following: respectively apply hierarchical clustering with average linkage, single linkage, and complete linkage to cluster subjects into groups, and create a dendrogram. For the dendrogram obtained from average linkage, find the height at which cutting the dendrogram gives the same number of groups in “`labels1`”, and comment on the clustering results obtained at this height by comparing them to the truth contained in “`labels1`”.

```
library(ggdendro)
source("Plotggdendro.r") #called to create the dendrogram with a line at the cut height
set.seed(123)
twofiftygenes_h = sample(colnames(stdgexpProj1), 250)
twofiftygeneset_h = stdgexpProj1[, twofiftygenes_h]

rownames(twofiftygeneset_h) = labels1 %>%
  filter(rownames(labels1) %in% rownames(twofiftygeneset_h)) %>%
  pull(Class)

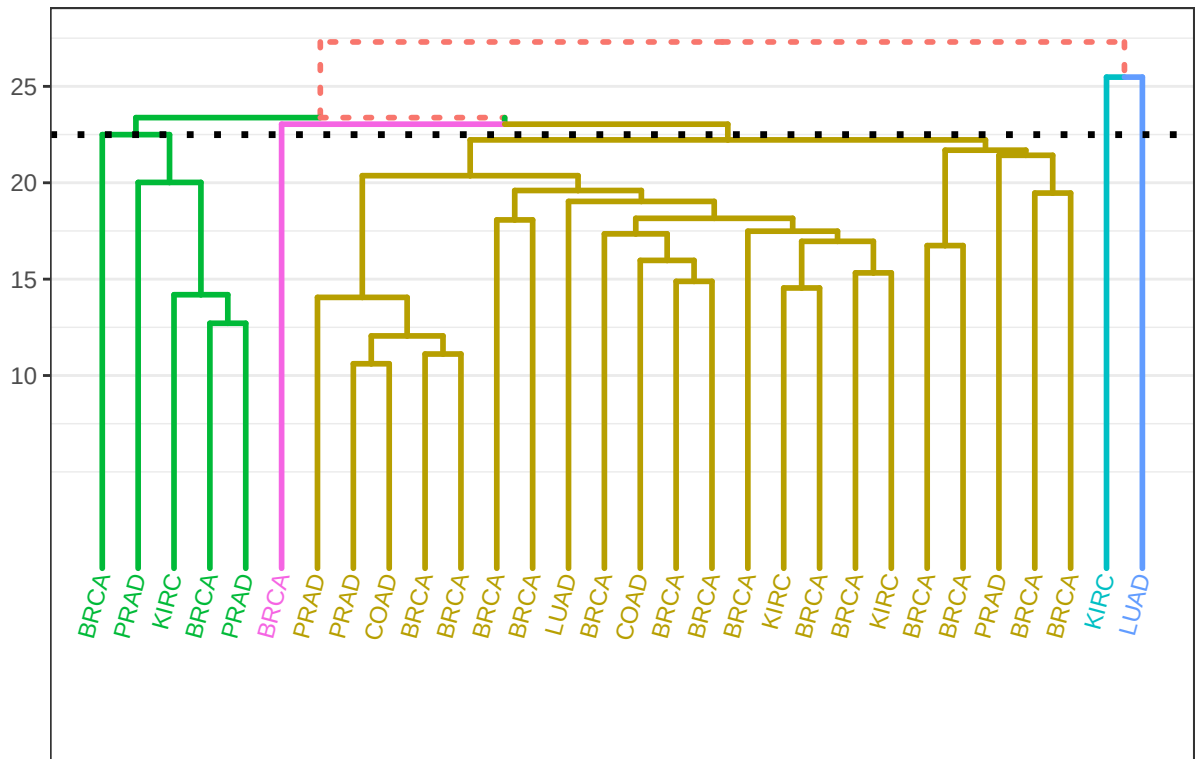
D = dist(twofiftygeneset_h)
hc_average = hclust(D, method = "average")
hc_single = hclust(D, method = "single")
hc_complete = hclust(D, method = "complete")

cutheight = hc_average$height[length(hc_average$height) - 4]
ave_dendro_data = dendro_data_k(hc_average, 5)

plot_ggdendro(ave_dendro_data, direction = "tb", heightReferece = cutheight,
  expand.y = 0.3) + ggtitle("Clustering with Average Linkage")
```

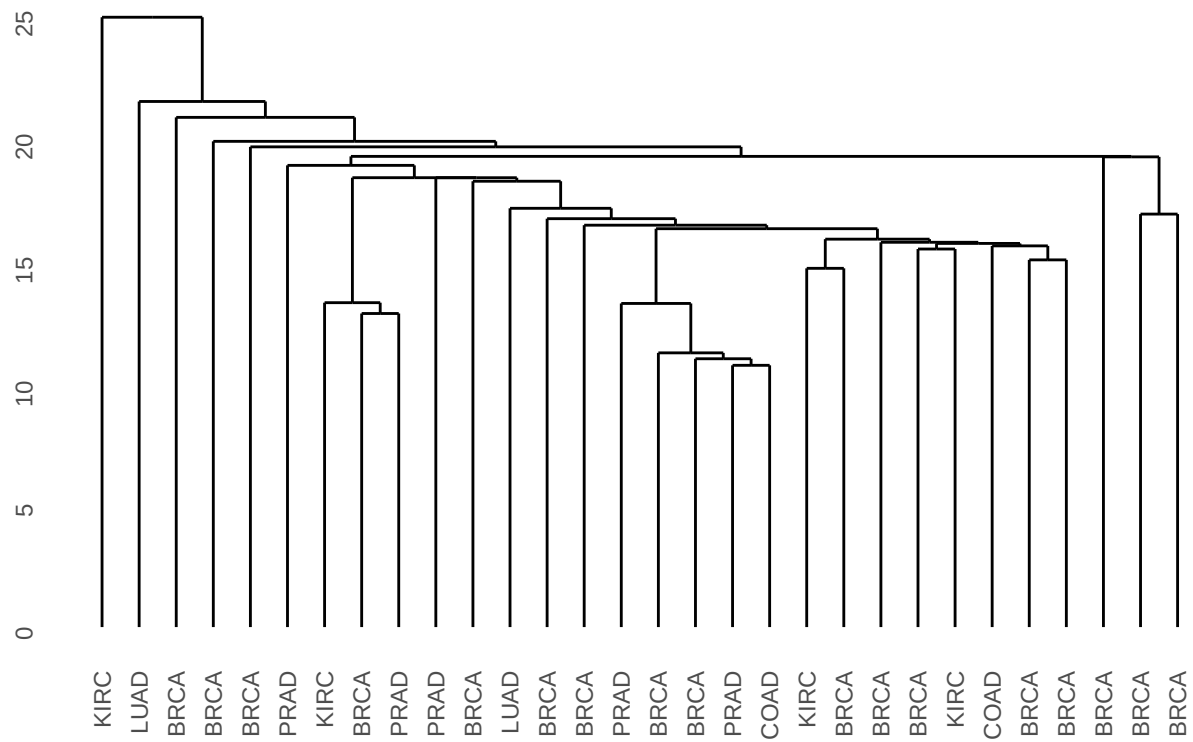
```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```


Clustering with Average Linkage



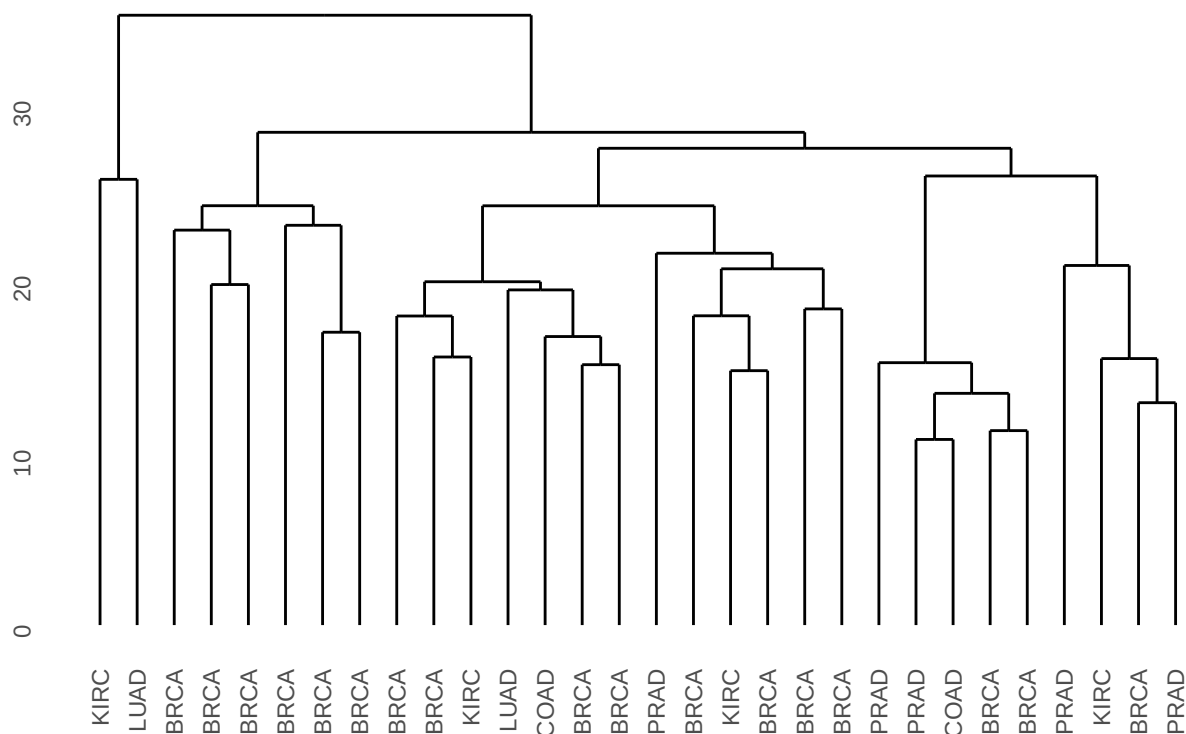
```
ggdendrogram(hc_single) + ggtitle("Hierarchical Clustering with Single Linkage")
```

Hierarchical Clustering with Single Linkage



```
ggdendrogram(hc_complete) + ggtitle("HC with complete linkage")
```

HC with complete linkage



```
table = table(ave_dendro_data$labels$label, ave_dendro_data$labels$clust)
correct_classifications = sum(diag(table))
total_classifications = sum(table)
classification_errors = total_classifications - correct_classifications
error_rate = classification_errors/total_classifications
print(table)
```

```
##
##      1  2  3  4  5
## BRCA 14  2  0  0  1
## COAD  2  0  0  0  0
## KIRC  2  1  1  0  0
## LUAD  1  0  0  1  0
## PRAD  3  2  0  0  0
```

```
cat("HC Clustering Accuracy:", 1 - error_rate, "%")
```

```
## HC Clustering Accuracy: 0.5333333 %
```

Examining the dendrogram created for Hierarchical Clustering using average linkage, we can compare the color of the generated clusters with the true labels which are listed on the bottom. With

this comparison in mind, it is fair to say this clustering based on 250 unique genes is not very accurate or conclusive. There are many mismatched colors and labels. Additionally 3 of the estimated clusters light blue, green, and pink all contain 3 or fewer observations yet contain labels that are seen in other clusters. Examining the dendrogram from a human perspective, there does not appear to be a clear pattern between the color (HC Predicted Label) and the true cancer class listed at the bottom. Examining the HC clustering accuracy

Task B. Classification

For this task, we will use the same data set you would have downloaded. Please use `set.seed(123)` for random sampling via the command `sample` and any other process where artificial randomization is needed.

(**Task B1**) After you obtain “labels.csv” and “data.csv”, do the following:

- Filter out genes (from “data.csv”) whose expressions are zero for at least 300 subjects, and save the filtered data as R object “gexp2”.
- Use the command `sample` to randomly select 1000 genes and their expressions from “gexp2”, and save the resulting data as R object “gexp3”.
- Pick the samples from “labels.csv” that are for cancer type “LUAD” or “BRCA”, and save them as object “labels2”. For these samples, pick the corresponding gene expressions from “gexp3” and save them as object “stdgexp2”

(**Taks B2**) The assumptions of linear or quadratic discriminant analysis requires that each observation follows a Gaussian distribution given the class or group membership of the observation, and that each observation follows a Gaussian mixture model. In our settings here, each observation (as a row) within a group would follow a Gaussian with dimensionality equal to the number of genes (i.e., number of entries of the row). So, the more genes whose expressions we use for classification, the higher the dimension of these Gaussian distributions. Nonetheless, you need to check if the Gaussian mixture assumption is satisfied. Note that we only consider two classes “LUAD” and “BRCA”, for which the corresponding Gaussian mixture has 2 components and hence has 2 bumps when its density is plotted.

Do the following and report your findings on classification:

- Randomly pick 3 genes and their expressions from “stdgexp2”, and save them as object “stdgexp2a”.
- Randomly pick 60% of samples from “stdgexp2a”, use them as the training set, and use the rest as the test set. You can round down the number of samples in the training set by the command `floor` if it is not an integer.

Build a quadratic discriminant analysis model using the training set, and apply the obtained model to the test set to classify each of its observations. You should code “BRCA” as 0 and “LUAD” as 1. If for an observation the posterior probability of being “BRCA” is predicted by the model to be greater than 0.5, the observation is classified as “BRCA”. Report via a 2-by-2 table on the

classification errors. Note that the predicted posterior probability given by `qda` is for an observation to belong to class “BRCA”.

Before building a quadratic discriminant analysis model, you need to check for highly correlated gene expressions, i.e., you need to check the sample correlations between each pair of columns of the training set. If there are highly correlated gene expressions, the estimated covariance matrix can be close to being singular, leading to unstable inference. You can remove a column from two columns when their contained expressions have sample correlation greater than 0.9 in absolute value.

(**Taks B3**) Do the following:

- Randomly pick 100 genes and their expressions from “stdgexp2”, and save them as object “stdgexp2b”.
- Randomly pick 75% of samples from “stdgexp2b”, use them as the training set, and use the rest as the test set. You can round down the number of samples in the training set by the command `floor` if it is not an integer.

Then apply quadratic discriminant analysis by following the requirements given in **Taks B2**. Compare classification results you find here with those found in **Taks B2**, and explain on any difference you find between the classification results.

(**Taks B4**) Do the following:

- Randomly pick 100 genes and their expressions from “stdgexp2”, and save them as object “stdgexp2b”.
- Randomly pick 75% of samples from “stdgexp2b”, use them as the training set, and use the rest as the test set. You can round down the number of samples in the training set by the command `floor` if it is not an integer.

Then apply k-nearest-neighbor (k-NN) method with neighborhood size $k=3$ to the test data to classify each observation in the test set into one of the cancer types. Here, for an observation, if the average of being cancer type “BRCA” is predicted by k-NN to be greater than 0.5, then the observation is classified as being “BRCA”. Report via a 2-by-2 table on the classification errors. Compare and comment on the classification results obtained here to those obtain in **Taks B3**. If there is any difference between the classification results, explain why.